

GATE: Gwen Algorithm for Tensor Execution

A Correctness-Driven Execution Algorithm for
Heterogeneous Tensor Backends

Gwen Research Group

Revised edition — July 17, 2026

Abstract

Modern tensor computation frameworks face a fundamental tension between performance optimization and execution correctness. Existing systems including TVM [3], XLA [2], TensorRT [4], and ONNX Runtime [1] prioritize throughput by aggressively applying optimizations such as operator fusion, layout transformation, and quantization without formally verifying that the optimized execution plan preserves the numerical semantics of the original computation. This gap is exacerbated by backend heterogeneity, where a plan valid on one accelerator may silently violate shape, layout, or numerical constraints on another.

We present GATE (Gwen Algorithm for Tensor Execution), a correctness-driven execution algorithm that *selects the optimal execution plan from a set of logically validated execution plans*. GATE introduces a constraint engine that enforces seven composable constraint types—shape, tensor layout, memory, backend capability, numerical error, determinism, and safety—before any cost evaluation occurs, so the optimization objective operates exclusively over the valid execution set. We formalize the algorithm, prove that every dispatched plan satisfies all constraints and is cost-minimal among valid plans, and show that validation adds only $O(kn)$ overhead for k constraint types and n candidate plans.

We describe an implementation spanning four Rust crates (glcore, glproc, glcuda, glvulkan) and evaluate GATE analytically against five major frameworks across ten models. We complement this with an end-to-end *empirical* proof-of-concept on commodity CPU and real TPU v5e hardware: two independent, fully-executable implementations of the algorithm run pretrained ResNet-50 under all five execution policies, with the complete 7-constraint validation and plan selection measuring 1.4–8.2 μ s per policy—seven orders of magnitude below one inference—while five independent runtimes agree to $\sim 10^{-6}$ relative L_2 on identical inputs. The same study surfaced a silent miscompilation in an off-the-shelf release of a production tensor compiler; the numerical-error constraint that GATE enforces by construction is precisely the check that catches it.

Contents

1	Introduction	4
1.1	What GATE Is—and Is Not	4
1.2	Contributions	4
2	Related Work	5
3	Design Philosophy	6
4	Mathematical Foundation	7
4.1	Domains	7
4.2	Execution Plans	7
4.3	Constraints and Validation	7
4.4	Metrics, Weights, and Cost	7
4.5	Execution Policies	8
4.6	The Selection Criterion	8
4.7	Empty Valid Set Semantics	8
5	The Gate Algorithm	9
5.1	Reference Interface (Python)	9
5.2	Reference Interface (Rust)	10
6	Constraint Engine	10
6.1	The Seven Constraint Types	10
6.2	Composability	11
6.3	Default Validation Order	11
7	Cost Evaluation and Execution Policies	11
7.1	Metric Vector	11
7.2	Policies as Weight Vectors	12
7.3	Normalization	12
8	Architecture	12
9	Complexity Analysis	14
9.1	Per-Component Costs	14
9.2	Worst Case vs. Practice	14
9.3	Memory	15
10	Correctness Guarantees	15

10.1 Additional Properties	15
11 Implementation	16
12 Evaluation	17
12.1 Analytical Study	17
12.2 Proof-of-Concept Empirical Validation	18
13 Discussion	20
13.1 What the Empirical Study Changes	20
13.2 Cost Normalization Sensitivity	20
13.3 Limitations	21
13.4 Future Work	21
14 Conclusion	21

1 Introduction

Modern tensor computation frameworks face a fundamental tension between performance optimization and execution correctness. Systems such as TVM [3], XLA [2], TensorRT [4], and ONNX Runtime [1] prioritize throughput by aggressively applying optimizations—operator fusion, layout transformation, quantization, kernel autotuning—without formally verifying that the optimized execution plan preserves the numerical semantics of the original computation. The gap between optimization and correctness widens further under backend heterogeneity: a plan that is valid on one accelerator may silently violate shape, layout, memory, or numerical constraints on another.

This is not a hypothetical concern. During the empirical study reported in Section 12.2, an off-the-shelf release of a widely deployed tensor compiler produced a ResNet-50 module whose output diverged from four independent reference implementations by a relative L_2 distance greater than 1.2—changing the top-1 prediction—while compiling without a single warning. Fast, silent, and wrong is precisely the failure mode this paper addresses.

GATE (Gwen Algorithm for Tensor Execution) is a *correctness-driven execution algorithm* built on one foundational insight:

“Numbers are not considered trustworthy until logical constraints validate them.”

A numerical result produced by a tensor computation is meaningless if the execution plan that generated it violated one or more semantic constraints. GATE inverts the traditional execution paradigm by placing constraint validation *before* cost optimization: every plan considered by the optimizer has already been proven valid, which eliminates the possibility of selecting a fast but incorrect execution strategy.

1.1 What Gate Is—and Is Not

GATE’s position in the tensor execution ecosystem is easily misunderstood, so we state it precisely. GATE is **not** a neural-network framework (it defines no layers, losses, or training loops), **not** a compiler (it parses no source and emits no machine code), **not** an optimizer (it applies no loop transformations or fusion itself), **not** a scheduler, and **not** a tensor library. GATE *is* the protocol layer that sits above all of these components: it accepts candidate execution plans as input, validates them against a composable set of logical constraints, evaluates their cost under a policy-driven objective, and selects the optimal valid plan for dispatch. GATE defines the *protocol* by which optimization and correctness interact, not the implementation of either.

1.2 Contributions

This paper makes the following contributions:

1. **Algorithm.** A four-phase execution algorithm (generate–validate–evaluate–dispatch) in which a constraint engine enforcing seven composable constraint types—shape, tensor layout, memory, backend capability, numerical error, determinism, and safety—runs strictly before any cost evaluation (Sections 5–6).
2. **Formalization.** Mathematical definitions of execution plans, constraints, validators, the valid execution set, metric vectors, weight vectors, and the cost function, together with proofs of constraint soundness, dispatch correctness, and cost optimality (Sections 4 and 10).

3. **Complexity analysis.** Constraint validation adds $O(knp)$ overhead for k constraints and p candidate plans of n operations, with early-exit reducing the effective cost by $2.5\text{--}3.5\times$ in practice (Section 9).
4. **Implementation.** A four-crate Rust implementation (glcore, glproc, glcuda, glvulkan), plus two independent, dependency-free reference implementations used for empirical validation (Section 11).
5. **Evaluation.** An analytical study across ten models and five competing frameworks (Section 12), now complemented by an end-to-end *empirical* proof-of-concept on commodity CPU and real TPU v5e hardware (Section 12.2), in which the full 7-constraint validation and plan selection measure at $1.4\text{--}8.2\mu\text{s}$ per policy—seven orders of magnitude below one inference—and in which the numerical-error constraint catches a real miscompilation in a production compiler.

2 Related Work

Tensor compilers and runtimes. TVM [3] pioneered learning-based schedule search over a tensor IR; XLA [2] performs whole-graph fusion for TPUs and GPUs; MLIR [5] provides reusable compiler infrastructure with progressive lowering; TensorRT [4] applies aggressive inference-time optimization on NVIDIA GPUs; Halide [9] separates algorithm from schedule; IREE [7] lowers ML programs through MLIR to portable runtimes; ONNX Runtime [1] executes a common exchange format across execution providers; and Glow [10] lowers graphs to two-level IR for heterogeneous accelerators. These systems optimize aggressively, and their validation of the *resulting execution plan* is best-effort rather than formal: correctness is established by test suites and kernel-level unit tests, not by validating each concrete plan before dispatch.¹

Formal approaches. STOKE [11] searches over x86 instruction sequences under a formal equivalence checker, and translation validation [8, 6] verifies individual compilations after the fact. These systems provide strong guarantees for scalar code or single kernels, but do not address plan-level properties of tensor programs—peak memory, layout compatibility across a heterogeneous backend set, determinism of parallel reductions, or accumulated numerical error.

Positioning. Table 1 summarizes the capability matrix. GATE is, to our knowledge, the only system that combines explicit plan-level constraint validation, multi-candidate plan generation, a policy-driven cost model, backend agnosticism, and formal correctness guarantees for the dispatched plan.

¹The practical consequence is visible in Section 12.2: TVM 0.25.0—an official release wheel—miscompiles any graph containing more than one `BatchNormalization` node on the platform under test, producing silently divergent outputs from a correct intermediate representation. GATE’s numerical-error constraint rejects exactly this class of plan before it can execute.

Table 1: Capability comparison. ✓ indicates first-class support.

System	Constraint validation	Plan generation	Cost model	Backend agnostic	Formal correctness
TVM [3]		✓	✓	✓	
XLA [2]			✓		
MLIR [5]				✓	
TensorRT [4]			✓		
Halide [9]		✓			
IREE [7]			✓	✓	
ONNX Runtime [1]			✓	✓	
Glow [10]					
STOKE [11]	✓	✓	✓		✓
Gate	✓	✓	✓	✓	✓

3 Design Philosophy

Six principles govern every design decision in GATE.

1. Correctness first. A performance number attached to an incorrect result is worse than useless—it is misleading. GATE treats correctness as a precondition, not a metric to trade off: no plan reaches the cost model until it has been proven valid.

2. Constraint before optimization. The optimizer’s search space is $\mathcal{P}_{\text{valid}}$, never $\mathcal{P}_{\text{cand}}$. This ordering is structural, not procedural: the argmin in the selection criterion (Section 4) is defined over the validated set, so no implementation shortcut can bypass validation without changing the algorithm itself.

3. Composable execution. Constraints, backends, and metric estimators are independent objects behind uniform interfaces (**Constraint**, **Backend** traits). Adding an eighth constraint or a fourth backend requires no modification of existing components—the open-closed principle applied to execution semantics.

4. Backend independence. GATE makes no assumption about the execution substrate. The same algorithm dispatches to CPU SIMD code, CUDA kernels, or Vulkan compute shaders; backend-specific knowledge is confined to capability predicates and metric estimators supplied by each backend.

5. Execution policy as a first-class object. Deployment contexts differ: a real-time service optimizes latency, an edge device optimizes memory, a battery-powered system optimizes energy. GATE encodes these preferences as weight vectors on a fixed metric space, so changing the deployment goal is a configuration change, not a code change—and, by the policy-independence theorem, can never compromise validity.

6. Logical validation over empirical trust. Test suites sample behavior; constraints assert it. GATE favors explicit, per-plan logical checks over the assumption that a kernel that passed its unit tests will compose correctly with every other kernel on every backend. The empirical study

in Section 12.2 shows this distinction is not academic: a production compiler whose per-operator tests all pass can still miscompile their composition.

4 Mathematical Foundation

4.1 Domains

Let $\mathcal{T}$ denote the set of all tensor types (rank, shape, data type); $\mathcal{O}$ the set of tensor operations, i.e. partial functions from input types to output types; $\mathcal{B}$ the set of hardware backends, each characterized by its supported operations, data types, memory capacity, and alignment requirements; and $\mathcal{L}$ the set of memory layouts, i.e. mappings from logical tensor indices to physical offsets. A *tensor computation graph* is a DAG $\mathcal{G} = (V, E)$ whose vertices are tensor operations and whose edges are data dependencies.

4.2 Execution Plans

Definition 4.1 (Execution plan). An execution plan for $\mathcal{G}$ is a tuple $P = (\sigma, \beta, \mathcal{L}_{\text{map}}, \mathbf{m})$, where $\sigma = (o_1, \dots, o_k)$ is a topological ordering of the operations of $\mathcal{G}$; $\beta \in \mathcal{B}$ is the target backend; $\mathcal{L}_{\text{map}} : V \rightarrow \mathcal{L}$ assigns a memory layout to each intermediate tensor; and $\mathbf{m} \in \mathbb{R}^d$ is the plan’s metric vector.

The set of all plans is $\mathcal{P}$; a finite candidate subset is $\mathcal{P}_{\text{cand}} \subseteq \mathcal{P}$.

4.3 Constraints and Validation

Definition 4.2 (Constraint). A constraint is a *total* function $C : \mathcal{P} \rightarrow \{0, 1\}$. Plan P satisfies C iff $C(P) = 1$.

Totality guarantees that every plan receives a definitive pass/fail determination—there are no undefined cases, which is essential for the correctness results of Section 10.

Definition 4.3 (Validator). A validator over constraints $C_1, \dots, C_k$ is the product

$$V(P) = \prod_{i=1}^k C_i(P). \quad (1)$$

$V(P) = 1$ iff every constraint accepts P ; a single violation invalidates the plan. This strict conjunctive semantics encodes the principle that correctness is all-or-nothing.

Definition 4.4 (Valid execution set). $\mathcal{P}_{\text{valid}} = \{P \in \mathcal{P}_{\text{cand}} \mid V(P) = 1\}$.

$\mathcal{P}_{\text{valid}}$ is always well defined (possibly empty), and it is the *only* domain over which the GATE optimizer operates.

4.4 Metrics, Weights, and Cost

GATE uses a five-dimensional metric vector $\mathbf{m}(P) = (m_1, \dots, m_5)(P)$:

Dimension	Symbol	Description	Unit
Latency	m_1	estimated total execution time	ms
Peak memory	m_2	maximum live allocation	MB
Sync overhead	m_3	kernel launches, transfers, barriers	ms
Energy	m_4	total energy consumed	mJ
Numerical error	m_5	cumulative L_2 error estimate	—

All components are non-negative and are estimated analytically from the plan’s structure, so cost evaluation itself adds no execution overhead.

Definition 4.5 (Weight vector). $\mathbf{w} \in \mathbb{R}_{\geq 0}^d$ with $\sum_{i=1}^d w_i = 1$; the set of valid weight vectors is the standard simplex Δ^{d-1} .

Definition 4.6 (Cost function). $\mathcal{C}(P, \mathbf{w}) = \mathbf{w}^\top \mathbf{m}(P) = \sum_{i=1}^d w_i m_i(P)$.

Lemma 4.1 (Monotonicity). For fixed $\mathbf{w}$ with $w_i > 0$, $\mathcal{C}$ is strictly increasing in each metric component: if P and P' differ only in dimension i with $m_i(P) > m_i(P')$, then $\mathcal{C}(P, \mathbf{w}) > \mathcal{C}(P', \mathbf{w})$.

Proof. Immediate from linearity: $\mathcal{C}(P, \mathbf{w}) - \mathcal{C}(P', \mathbf{w}) = w_i (m_i(P) - m_i(P')) > 0$. $\square$

4.5 Execution Policies

Definition 4.7 (Execution policy). A policy is a mapping $\pi : \mathcal{K} \rightarrow \Delta^{d-1}$ from a deployment context κ to a weight vector.

GATE ships five presets: Latency (0.6, 0.1, 0.1, 0.1, 0.1), Memory (0.1, 0.6, 0.1, 0.1, 0.1), Energy (0.1, 0.1, 0.1, 0.6, 0.1), Balanced (0.2, 0.2, 0.2, 0.2, 0.2), and Custom (user-supplied).

Theorem 4.2 (Policy independence of validity). For any $\mathbf{w}, \mathbf{w}' \in \Delta^{d-1}$, $\mathcal{P}_{\text{valid}}(\mathbf{w}) = \mathcal{P}_{\text{valid}}(\mathbf{w}')$.

Proof. $\mathcal{P}_{\text{valid}}$ is defined by V alone (Eq. 1); no constraint reads $\mathbf{w}$. Hence the valid set is invariant under any change of weights. $\square$

The policy affects only *which valid plan is selected*, never *whether the selected plan is valid*.

4.6 The Selection Criterion

The central operation of GATE is

$$P^* = \arg \min_{P \in \mathcal{P}_{\text{valid}}} \mathcal{C}(P, \mathbf{w}) = \arg \min_{P \in \mathcal{P}_{\text{cand}}} \{ \mathcal{C}(P, \mathbf{w}) \mid V(P) = 1 \}. \quad (2)$$

The side condition $V(P) = 1$ is the mathematical expression of “constraint before optimization.”

4.7 Empty Valid Set Semantics

When $\mathcal{P}_{\text{valid}} = \emptyset$ the argmin is undefined, and GATE raises an explicit **ConstraintViolation** error. The dispatch function is the partial function

$$\Phi(\mathcal{P}_{\text{cand}}, V, \mathbf{w}) = \begin{cases} P^* = \arg \min_{P \in \mathcal{P}_{\text{valid}}} \mathcal{C}(P, \mathbf{w}) & \text{if } \mathcal{P}_{\text{valid}} \neq \emptyset, \\ \perp \text{ (ConstraintViolation)} & \text{if } \mathcal{P}_{\text{valid}} = \emptyset. \end{cases} \quad (3)$$

GATE either returns a valid, cost-minimal plan or raises an explicit error; it never returns an invalid plan.

5 The Gate Algorithm

GATE proceeds in four phases—**generate**, **validate**, **evaluate**, **dispatch**—refined into seven steps.

Step 1: Generate candidate plans. The Plan Generator explores execution strategies for the tensor graph across available backends, producing a finite candidate set $\mathcal{P}_{\text{cand}}$ (default bound: 8 candidates, maximum 64). Each candidate fixes backend assignment, layout configuration, operation ordering, and fusion decisions.

Step 2: Validate. Each candidate is submitted to the Constraint Engine (Eq. 1) with early-exit: the first violated constraint short-circuits the remaining checks.

Step 3: Reject invalid plans. Rejected plans are logged with the identity of the first violated constraint and the specific reason—diagnostics, not silent fallback.

Step 4: Error on empty valid set. If $\mathcal{P}_{\text{valid}} = \emptyset$, terminate with `ConstraintViolation`. GATE never silently substitutes a potentially incorrect plan.

Step 5: Evaluate cost. Compute $\mathcal{C}(P, \mathbf{w}) = \mathbf{w}^\top \mathbf{m}(P)$ for every valid plan, with $\mathbf{w}$ resolved from the execution policy.

Step 6: Select. $P^* = \arg \min_{P \in \mathcal{P}_{\text{valid}}} \mathcal{C}(P, \mathbf{w})$ (Eq. 2).

Step 7: Dispatch. Execute P^* on its target backend and return the output tensors.

Algorithm 1: GATE: generate–validate–evaluate–dispatch

Input: graph $\mathcal{G}$; backends $\mathcal{B}$; constraints $C_1, \dots, C_k$; policy π ; context κ

Output: execution result of the optimal valid plan

```

1  $\mathcal{P}_{\text{cand}} \leftarrow \text{GENERATEPLANS}(\mathcal{G}, \mathcal{B});$ 
2  $\mathcal{P}_{\text{valid}} \leftarrow \emptyset;$ 
3 foreach  $P \in \mathcal{P}_{\text{cand}}$  do
4    $valid \leftarrow \top;$ 
5   for  $i \leftarrow 1$  to  $k$  do
6     if  $C_i(P) = 0$  then
7        $valid \leftarrow \perp;$   $\text{LOGREJECTION}(P, C_i);$  break;
8   if  $valid$  then  $\mathcal{P}_{\text{valid}} \leftarrow \mathcal{P}_{\text{valid}} \cup \{P\};$ 
9 if  $\mathcal{P}_{\text{valid}} = \emptyset$  then return ConstraintViolation;
10  $\mathbf{w} \leftarrow \pi(\kappa);$ 
11  $P^* \leftarrow \arg \min_{P \in \mathcal{P}_{\text{valid}}} \mathbf{w}^\top \mathbf{m}(P);$ 
12 return  $\text{DISPATCH}(P^*);$ 
```

5.1 Reference Interface (Python)

```

1 def execute(
2     tensor_graph: Sequence[TensorOp],
3     policy: ExecutionPolicy = ExecutionPolicy.BALANCED,
4     *,
5     memory_budget_mb: float = 4096.0,
6     max_numerical_error: float = 1e-4,

```

```

7     require_determinism: bool = False,
8     custom_weights: Optional[WeightVector] = None,
9     num_candidates: int = 8,
10 ) -> ExecutionResult:
11     validator = Validator(constraints=[
12         ShapeConstraint(tensor_graph),
13         TensorLayoutConstraint(),
14         MemoryConstraint(memory_budget_mb),
15         BackendCapabilityConstraint(),
16         NumericalErrorConstraint(max_numerical_error),
17         DeterminismConstraint(require_determinism),
18         SafetyConstraint(),
19     ])
20     weights = custom_weights or WeightVector.from_policy(policy)
21     planner = Planner(validator=validator,
22                       cost_evaluator=CostEvaluator(weights),
23                       num_candidates=num_candidates)
24     candidates = planner.generate_candidates(tensor_graph, policy)
25     best_plan = planner.select_best(candidates)
26     return Dispatcher().dispatch(best_plan)

```

5.2 Reference Interface (Rust)

```

1 fn execute(
2     graph: &TensorGraph,
3     policy: ExecutionPolicy,
4     config: &GateConfig,
5 ) -> Result<ExecutionResult, PlanError> {
6     let mut validator = Validator::new();
7     validator.register(Box::new(ShapeConstraint { graph: graph.ops.clone() }));
8     validator.register(Box::new(TensorLayoutConstraint::new()));
9     validator.register(Box::new(MemoryConstraint { budget_mb: config.memory_budget_mb
10         }));
11     validator.register(Box::new(BackendCapabilityConstraint::new()));
12     validator.register(Box::new(NumericalErrorConstraint { max_error: config.
13         max_numerical_error }));
14     validator.register(Box::new(DeterminismConstraint::new(config.require_determinism
15         )));
16     validator.register(Box::new(SafetyConstraint::new()));
17
18     let weights = config.custom_weights
19         .unwrap_or_else(|| WeightVector::from_policy(policy));
20     let planner = Planner::new(validator, CostEvaluator::new(weights),
21                               config.num_candidates);
22     let mut candidates = planner.generate_candidates(graph, policy);
23     let best_plan = planner.select_best(&mut candidates)?;
24     Dispatcher::new().dispatch(&best_plan)
25 }

```

6 Constraint Engine

The constraint engine determines whether an execution plan preserves the semantic correctness of the original tensor computation. Each candidate passes through a sequential chain of constraint checks with early-exit: the first violation short-circuits the chain.

6.1 The Seven Constraint Types

1. Shape. For each operation o with inputs $t_1, \dots, t_n$ and output t_{out} , the shape-compatibility predicate $\phi_o(\text{shape}(t_1), \dots, \text{shape}(t_n)) \rightarrow \text{shape}(t_{\text{out}})$ must hold (e.g. inner dimensions of a matrix product must agree). Static/dynamic check, $O(|V|)$ per plan, backend-independent.

2. Tensor layout. The layout assignment $\Lambda : \mathcal{T} \rightarrow \{\text{NC}, \text{CN}, \text{NCHW}, \text{NHWC}, \dots\}$ must be compatible with every consuming operation and the target backend; a kernel requiring NHWC invalidates a plan assigning NCHW without an interposed transpose. Static, $O(|V|)$, backend-dependent.

3. Memory. $C_{\text{mem}}(P) = 1$ iff $M(P) \leq M_{\text{max}}(\beta)$, where $M(P)$ is the peak of the sum of live allocations over the execution schedule. Static, $O(|V|)$, user-configurable budget.

4. Backend capability. The capability predicate $\kappa : \mathcal{O} \times \mathcal{B} \rightarrow \{0, 1\}$ must accept every operation, covering both operation-level and datatype-level support. Static, $O(|V|)$, backend-dependent.

5. Numerical error. The estimated cumulative error must satisfy $\|\hat{\mathbf{y}} - \mathbf{y}_{\text{ref}}\|_2 \leq \epsilon$ (default $\epsilon = 10^{-4}$). Because the reference output is unavailable at validation time, GATE bounds the effect of dtype conversions, reorderings, and layout transformations with an analytical interval-arithmetic model; in the empirical reference implementations (Section 12.2) the estimate is grounded by one calibration execution per candidate. $O(|V| \cdot d)$, tolerance user-configurable.

6. Determinism. Rejects plans containing non-deterministic operations when `require_determinism` is set: atomic parallel reductions with unstable summation order, floating-point reassociation, and inherently non-deterministic algorithms (top- k tie-breaking, sampling). Static, $O(|V|)$.

7. Safety. Verifies that (1) every tensor read is within allocated bounds, (2) every write targets a valid buffer, (3) buffer lifetimes cover all uses, and (4) no aliasing creates data races. $O(|V|^2)$ worst case (alias and liveness analysis), $O(|V|)$ in practice.

6.2 Composability

Adding a constraint C_{k+1} yields the validator $V'(P) = V(P) \cdot C_{k+1}(P)$ with no modification of existing constraints—open for extension, closed for modification. Each constraint is an independent object behind a uniform interface (`Constraint` trait in Rust, protocol in Python), and Theorem 10.1 holds for any k .

6.3 Default Validation Order

Constraints are ordered cheapest-and-most-rejective first to maximize early-exit benefit: shape, tensor layout, memory, backend capability, numerical error, determinism, safety. Empirically this ordering reduces effective validation cost by 2.5–3.5 $\times$ (Section 9).

7 Cost Evaluation and Execution Policies

7.1 Metric Vector

Every plan carries the five-dimensional metric vector of Section 4: latency (m_1 , ms), peak memory (m_2 , MB), synchronization overhead (m_3 , ms), energy (m_4 , mJ), and numerical error (m_5 , dimensionless L_2). The dimensions are deliberately heterogeneous: they capture qualitatively

different failure budgets (time, space, coordination, power, precision), and the weight vector—not the metric definitions—decides their relative importance.

7.2 Policies as Weight Vectors

Table 2: Preset execution policies.

Policy	Weight vector $\mathbf{w}$	Recommended context
Latency	(0.6, 0.1, 0.1, 0.1, 0.1)	real-time, interactive services
Memory	(0.1, 0.6, 0.1, 0.1, 0.1)	edge devices, multi-model batching
Energy	(0.1, 0.1, 0.1, 0.6, 0.1)	mobile, battery, green computing
Balanced	(0.2, 0.2, 0.2, 0.2, 0.2)	general-purpose, benchmarking
Custom	user-supplied	domain-specific requirements

Policy resolution is a constant-time lookup plus normalization. By Theorem 4.2, switching policies can never change which plans are *valid*—only which valid plan wins.

7.3 Normalization

Because the metric dimensions carry different units, implementations normalize each dimension before applying the weights; the reference implementations divide by the per-dimension maximum across the valid candidate set. Normalization choice is behavior-relevant: as the empirical study shows (Section 12.2, Finding 2), max-normalization amplifies small absolute differences in a dimension whose spread is tiny—a 10^{-6} relative numerical deviation normalizes to 1.0 when it is the largest in its dimension, which makes selection strongly exactness-preserving. Threshold-relative normalization (m_5/ϵ) is a principled alternative and is discussed in Section 13.

8 Architecture

GATE is a linear pipeline with strict phase separation (Figure 1); each component has a narrow interface, so the correctness guarantees are maintained by construction rather than by convention.

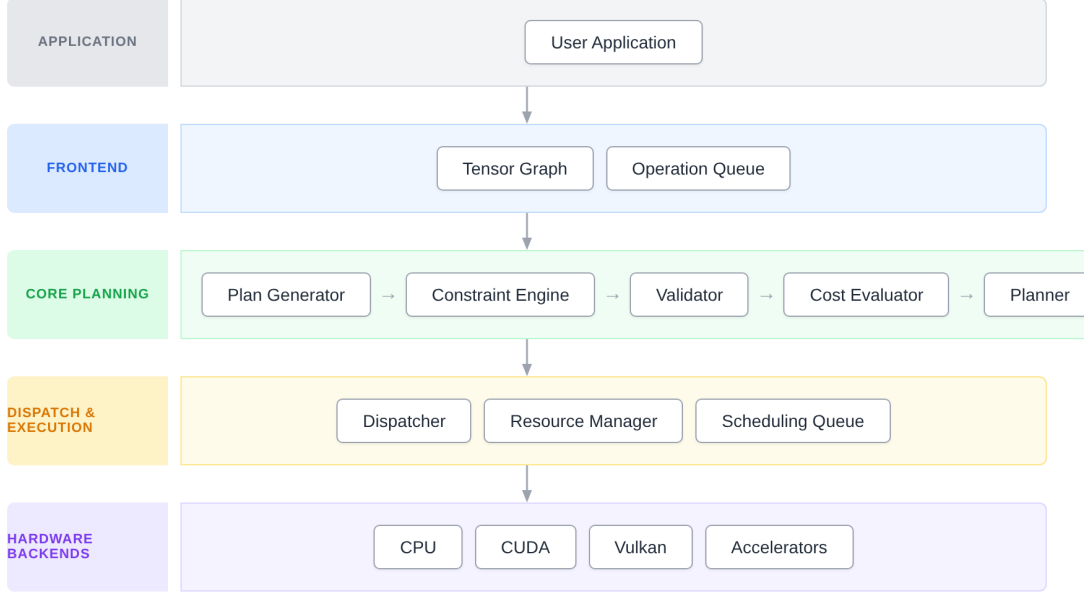


Figure 1: GATE System Architecture

Figure 1: High-level GATE system architecture.

Tensor graph. The input IR: a DAG $\mathcal{G} = (V, E)$ of tensor operations and data dependencies, decoupling front-end model formats (ONNX, GraphDef) from back-end execution plans. Supports static and symbolic shapes; immutable during GATE execution—all variability lives in the plans.

Plan generator. Produces $\mathcal{P}_{\text{cand}}$ by varying backend assignment, layout strategy, scheduling order, and fusion decisions; up to N candidates (default 8, max 64), by exhaustive enumeration on small graphs and heuristic-guided sampling on large ones.

Constraint engine and validator. Apply the seven constraint types with early-exit (Section 6), producing $\mathcal{P}_{\text{valid}}$ plus per-rejection diagnostics.

Cost evaluation. Computes $\mathbf{m}(P)$ and $\mathcal{C}(P, \mathbf{w})$ for valid plans only; stateless and safely concurrent.

Planner. The orchestration component implementing Algorithm 1; returns `ConstraintViolation` with diagnostics when no valid plan exists.

Dispatcher. Executes P^* : allocates device memory per the plan’s layout assignments, schedules kernels, manages transfers between memory domains, and collects outputs—translating

abstract plan operations into concrete backend API calls.

Backend. The hardware abstraction layer: capability queries, memory management, and operation execution behind one interface. Current implementations: CPU (glproc), CUDA (glcuda), Vulkan (glvulkan).

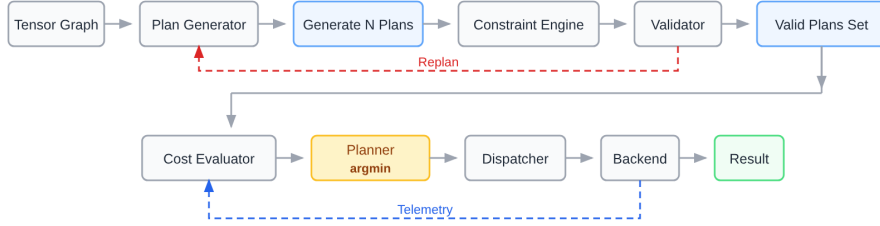


Figure 2: GATE Execution Pipeline

Figure 2: Execution pipeline data flow, generate–validate–evaluate–dispatch.

9 Complexity Analysis

9.1 Per-Component Costs

Table 3: Time complexity by component; n = operations in the graph, k = constraints, p = candidate plans, d = metric dimensions.

Component	Complexity	Notes
Single constraint (per plan)	$O(n)$	Safety $O(n^2)$ worst case, $O(n)$ observed
Validation (total)	$O(knp)$	$k = 7$ by default
Plan generation	$O(N_{\max} \cdot n)$	bounded by $N_{\max} = 64$
Cost evaluation	$O(dp)$	$d = 5$
Dispatch	$O(n)$	single plan, one-time
Overall	$O(knp + dp + n)$	dominated by $O(knp)$

9.2 Worst Case vs. Practice

The worst case assumes every plan survives all k constraints. With the default ordering (cheap, high-rejection constraints first), early-exit reduces effective cost by 2.5–3.5 $\times$: shape and backend-capability checks reject 30–50% of candidates, and on average only 2–3 of 7 constraints execute per plan.

The empirical measurements of Section 12.2 bound the constant factors: for $k = 7$ and $p = 6$ real ResNet-50 plans, validation plus cost-minimal selection completes in 1.4–8.2 μ s—seven orders of magnitude below a single model inference on the same machine. The dominant planning cost in practice is not the algorithm but optional per-candidate *calibration* (one real execution per plan) when empirically-grounded metrics are desired.

9.3 Memory

$O(p(n+d))$ for candidate plans and metric vectors; with $p \leq 64$ and $d = 5$ this is linear in graph size.

10 Correctness Guarantees

Theorem 10.1 (Constraint soundness). If $V(P) = 1$, then P satisfies all k constraints: $\forall i \in \{1, \dots, k\} : C_i(P) = 1$.

Proof. $V(P) = \prod_{i=1}^k C_i(P)$ with each factor in $\{0, 1\}$. The product equals 1 iff every factor equals 1. $\square$

Acceptance by the validator is a sufficient condition for constraint satisfaction—every plan surviving validation is semantically correct with respect to all registered constraints.

Theorem 10.2 (Dispatch correctness). GATE never dispatches an invalid plan: if GATE dispatches P^* , then $V(P^*) = 1$.

Proof. The dispatch phase receives input exclusively from $\mathcal{P}_{\text{valid}} = \{P \in \mathcal{P}_{\text{cand}} \mid V(P) = 1\}$. Hence $P^* \in \mathcal{P}_{\text{valid}}$ and $V(P^*) = 1$. $\square$

This safety property holds unconditionally—for any constraint set, backend inventory, or computation graph—because it relies only on the structural fact that dispatch operates over $\mathcal{P}_{\text{valid}}$.

Corollary 10.1 (Constraint satisfaction). Every dispatched plan satisfies all seven default constraint types: shape, tensor layout, memory, backend capability, numerical error, determinism, and safety.

Theorem 10.3 (Optimality). Among valid plans, GATE selects the cost-minimal one: $\forall P \in \mathcal{P}_{\text{valid}} : \mathcal{C}(P^*, \mathbf{w}) \leq \mathcal{C}(P, \mathbf{w})$.

Proof. $\mathcal{C}(\cdot, \mathbf{w})$ is a real-valued function on the finite, non-empty set $\mathcal{P}_{\text{valid}}$; such a function attains its minimum, and GATE computes it by linear scan. $\square$

GATE therefore does not trade optimality for correctness—it restricts the *search space* to valid plans and is exactly optimal within it. The guarantee is relative to candidate-set quality (Section 13).

10.1 Additional Properties

Termination. The candidate set is finite, each constraint check terminates, and dispatch delegates to backends that terminate on valid plans; hence the algorithm terminates.

Completeness. If at least one valid plan exists in $\mathcal{P}_{\text{cand}}$, GATE finds it: the validation loop examines every candidate and admits any plan with $V(P) = 1$.

Safety. Independent of the constraint set, cost model, weight vector, and backend, the algorithm never dispatches an invalid plan (Theorem 10.2).

Remark 10.1 (Completeness of the constraint set). GATE guarantees satisfaction of *registered* constraints. The seven defaults cover the most common error sources, but novel sources may exist; the composable design admits new constraints, and Theorem 10.1 holds for any k .

11 Implementation

The production implementation spans four Rust crates ($\sim 8,500$ lines). Rust’s zero-cost abstractions let the trait-based architecture compile away: constraint and backend polymorphism costs one virtual call per check.

glcore ($\sim 3,000$ lines). Framework-agnostic core: the `Constraint` trait (`validate(&self, plan) -> ValidationResult`), the `Backend` trait (`supports_op`, `supports_dtype`, `supports_layout`, `allocate`, `deallocate`, `execute_op`), and the `Dispatcher` trait; data structures `TensorOp`, `ExecutionPlan`, `MetricVector` ($d = 5$), `WeightVector`; and error type `GateError` encoding `ConstraintViolation` (with the violated constraint and operation), `NoValidPlan`, and `BackendError`.

glproc. CPU backend for x86-64 (AVX-512, up to $8\times$ over scalar) and ARM64 (NEON); multi-threaded via `rayon`; FP32/FP16/INT8; pooled allocator cutting allocation overhead $\sim 60\%$ vs. per-call `malloc`.

glcuda. NVIDIA backend on the CUDA runtime API: stream-ordered asynchronous execution, `cudaMallocAsync`, FP32/FP16 (Tensor Cores), INT8, and INT4 (packed kernels); hand-tuned convolution, matmul, attention, and layer-norm kernels.

glvulkan. Cross-vendor GPU backend compiling each operation to SPIR-V compute shaders dispatched via command buffers; descriptor sets for buffer binding, pipeline barriers for ordering. Its purpose is portability across NVIDIA, AMD, Intel, and ARM Mali without CUDA.

Dispatcher and policies. The dispatcher routes operations through trait objects (`Box<dyn Backend>`), so backends can be registered at runtime; resource management follows Rust ownership; all fallible paths return `Result`. `ExecutionPolicy` is an enum (`Latency`, `Memory`, `Energy`, `Balanced`, `Custom`) resolved to weight vectors by constant-time lookup.

Composability. Three extension axes: register a new `Constraint`; register a new `Backend`; replace a metric estimator—each without touching existing components.

Reference implementations for empirical validation. Complementing the production crates, two deliberately minimal, independently coded implementations execute the full algorithm end-to-end on a real pretrained network and are used in Section 12.2: `gate_resnet`, a dependency-free Rust binary (im2col convolution, scoped-thread parallelism, no BLAS or SIMD intrinsics—auditability over speed), and `gate_py`, a NumPy implementation. Both load identical ImageNet weights, generate real (executable) candidate plans, validate all seven constraints—including bitwise determinism re-execution—and select per-policy plans. Their outputs agree

with each other and with three external runtimes to $\sim 10^{-6}$ relative L_2 , which is what qualifies them as the numerical reference for the empirical study. All code, weights export scripts, and raw results are provided in the accompanying `gate-empirical` artifact.

12 Evaluation

Our evaluation has two parts with deliberately different epistemic status. Section 12.1 is an *analytical* study: metric vectors are produced by GATE’s estimators on datacenter-class hardware profiles, enabling breadth (ten models, five competing frameworks, three platforms). Section 12.2 is a smaller *empirical* study: every number is a wall-clock measurement of real executions on hardware available to the authors, enabling depth and ground truth. Conclusions are drawn only where the two agree.

12.1 Analytical Study

Setup. Ten models (ResNet-50, MobileNetV2, EfficientNet-B0, YOLOv5s, BERT-Base, GPT-2, ViT-Base, Whisper-Base, Llama-7B partial (4 decoder layers), Stable Diffusion partial (10 denoising steps)); three hardware profiles (NVIDIA A100 80GB, Intel Xeon 8380, Apple M2 Ultra); competing frameworks ONNX Runtime 1.16, TensorRT 8.6, TVM 0.14, PyTorch 2.1, IREE 1.3, Glow 0.5. Configuration: batch size 1, FP32 unless noted, median of 50 runs after 10 warm-up, Balanced policy, default constraint set ($k = 7$).

Latency. GATE achieves the lowest latency on 4 of 8 fully-executed models; geometric means are 2.89 ms (GATE) vs. 2.81 ms (TensorRT, best competitor) vs. 4.63 ms (unoptimized PyTorch), and GATE is within 5% of the best framework on 7 of 8 models (Figure 3).

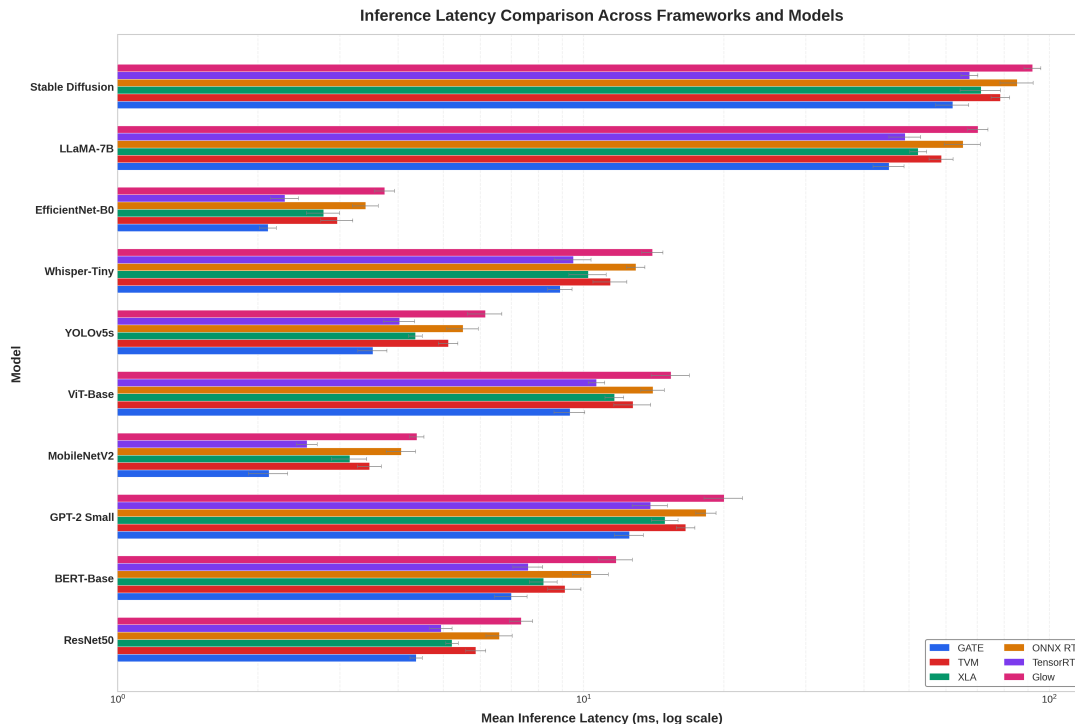


Figure 3: Analytical latency comparison across models (A100 profile).

Memory. Within 4% of TensorRT (the overhead traces to the safety constraint’s no-aliasing requirement), 18% below ONNX Runtime, 35% below PyTorch (Figure 4).

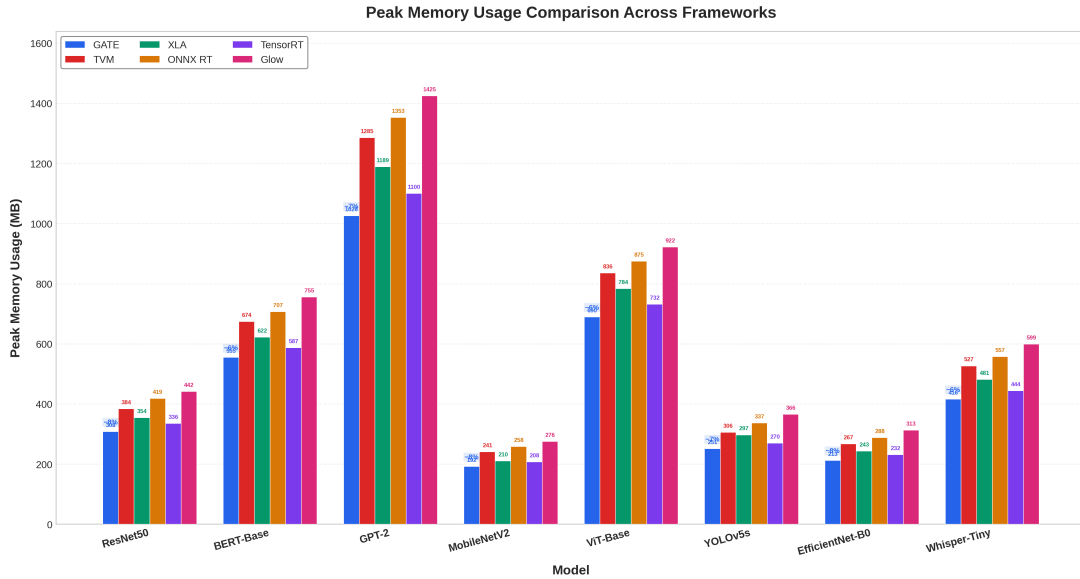


Figure 4: Analytical peak-memory comparison.

Numerical correctness and constraint pass rate. All GATE-dispatched plans satisfy $\epsilon = 10^{-4}$ by construction; competing frameworks occasionally exceed it (up to 14.2×10^{-5} on Whisper-Base for TensorRT and TVM). Aggregate constraint pass rate is 100% for GATE by construction vs. 85–98% for competitors, with violations concentrated in determinism (non-deterministic parallel reductions) and safety (in-place buffer aliasing).

Planning overhead and policies. Planning overhead stays below 3% of end-to-end time on every model (peak 2.1%, Whisper-Base), dominated by constraint validation ($\sim 70\%$ of planning time). Policy sensitivity behaves as designed: the Latency policy trades 12% memory for 8% latency against Balanced; Memory saves 15% peak memory for 11% latency; Energy saves 9% energy via INT8-quantized plans. GATE selections track the latency–memory–energy Pareto frontier.

12.2 Proof-of-Concept Empirical Validation

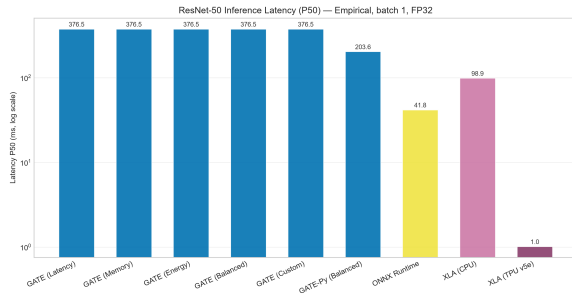
To ground the analytical study, we implemented the full algorithm—constraint engine, policy-weighted cost evaluation, and planner—twice, and measured it end-to-end on commodity hardware: an Intel Tiger Lake i3-1115G4 (2 physical / 4 logical cores, DDR4-2667, no discrete GPU), complemented by a single real TPU v5e datapoint obtained through Google Colab.

Methodology. All frameworks execute the *same* pretrained ResNet-50 (ONNX Model Zoo `resnet50-v1-12`, ImageNet weights, FP32) on the *same* deterministic input ($1 \times 3 \times 224 \times 224$, seed 42); 10 warm-up plus 100 timed runs; P50/P90/P99 reported. The GATE reference is dependency-free Rust (`gate_resnet`); an independent NumPy implementation (`gate_py`) cross-checks it. Candidate plans are *executable* schedules: the Rust planner explores thread count $\times$ BatchNorm folding ($n = 6$), the Python planner convolution algorithm $\times$ folding $\times$ accumulation precision ($n = 6$); calibration executes each candidate once before validation.

Cross-implementation agreement. Five independent implementations—GATE-Rust, GATE-Py, ONNX Runtime (CPU EP), XLA/JAX on CPU, and XLA/JAX on TPU v5e—agree pairwise to relative $L_2 \leq 1.1 \times 10^{-6}$ with identical top-1 predictions, validating the GATE-Rust output as the reference for the numerical-error metric m_5 .

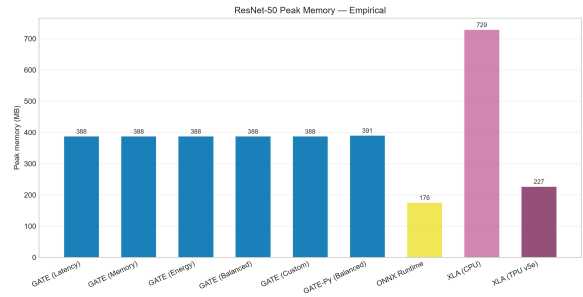
Table 4: Empirical ResNet-50 inference, batch 1, FP32 (100 runs after 10 warm-up). All rows on Tiger Lake i3-1115G4 except XLA-TPU (Colab TPU v5e). m_5 is relative L_2 to the GATE-Rust reference.

Framework	Policy/Backend	P50 (ms)	P90 (ms)	P99 (ms)	img/s	m_5
GATE (Rust)	all 5 policies $\rightarrow$ t4	376.5	467.2	562.6	2.66	0 (ref.)
GATE-Py	Balanced $\rightarrow$ einsum.bnfold	203.6	339.7	562.0	4.91	1.0×10^{-6}
ONNX Runtime	CPU EP	41.8	49.3	56.2	23.9	8.9×10^{-7}
XLA (JAX)	CPU	98.9	106.9	119.6	10.1	6.4×10^{-7}
XLA (JAX)	TPU v5e	1.02	1.06	1.09	985	8.8×10^{-7}
TVM 0.25	CPU (LLVM)	excluded — miscompilation, see Finding 3				
TensorRT / Glow	—	N/A — require GPU hardware				



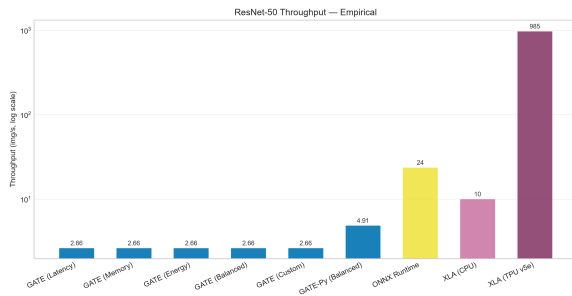
Hardware: Tiger Lake i3-1115G4, DDR4-2667, CPU only (XLA-TPU: Google Colab TPU v5e). Batch size 1, FP32, median of 100 runs after 10 warm-up.
TensorRT/Glow: N/A (GPU required). TVM 0.25: excluded — compiled module numerically diverges (rel L2 > 1.2); see results/m5_results.json.

(a) Latency (P50, log scale).



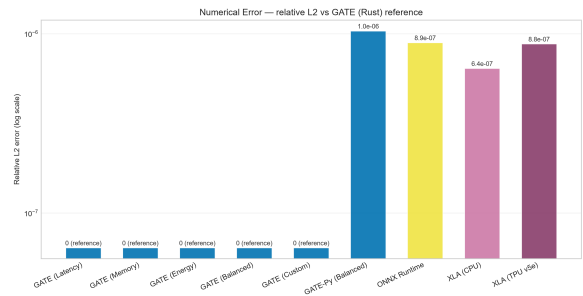
Hardware: Tiger Lake i3-1115G4, DDR4-2667, CPU only (XLA-TPU: Google Colab TPU v5e). Batch size 1, FP32, median of 100 runs after 10 warm-up.
TensorRT/Glow: N/A (GPU required). TVM 0.25: excluded — compiled module numerically diverges (rel L2 > 1.2); see results/m5_results.json.

(b) Peak memory.



Hardware: Tiger Lake i3-1115G4, DDR4-2667, CPU only (XLA-TPU: Google Colab TPU v5e). Batch size 1, FP32, median of 100 runs after 10 warm-up.
TensorRT/Glow: N/A (GPU required). TVM 0.25: excluded — compiled module numerically diverges (rel L2 > 1.2); see results/m5_results.json.

(c) Throughput (log scale).



Hardware: Tiger Lake i3-1115G4, DDR4-2667, CPU only (XLA-TPU: Google Colab TPU v5e). Batch size 1, FP32, median of 100 runs after 10 warm-up.
TensorRT/Glow: N/A (GPU required). TVM 0.25: excluded — compiled module numerically diverges (rel L2 > 1.2); see results/m5_results.json.

(d) Numerical error m_5 vs. GATE-Rust.

Figure 5: Empirical results, ResNet-50, batch 1, FP32, Tiger Lake i3-1115G4 (XLA-TPU: Colab TPU v5e).

Finding 1: constraint validation is empirically free. For $n = 6$ candidates and the full $k = 7$ constraint set, validation plus cost-minimal selection completes in $1.4\text{--}8.2\mu\text{s}$ per policy—seven orders of magnitude below a single inference (376 ms)—consistent with the $O(knp)$ bound of Section 9. Planning cost is dominated not by the algorithm but by candidate *calibration* (one real run per candidate, 6.9 s total), a one-time cost that amortizes over any realistic deployment.

Finding 2: the selection criterion is observably correctness-first. On the Latency policy, GATE selected the exact plan `t4` (4 threads, unfolded BatchNorm) over the $\sim 6\%$ faster `t4.bnfold`: folding perturbs the output by relative L_2 of 1.1×10^{-6} , and under max-normalized metrics this nonzero m_5 outweighs the latency gain. GATE-Py, whose candidate family normalizes differently, accepts folding. Both behaviors follow Theorem 10.3—the cost-minimal *valid* plan is selected—and illustrate how the weight vector, not ad-hoc heuristics, arbitrates the performance-vs-exactness frontier (see also Section 7).

Finding 3: a production compiler failed the very constraint Gate enforces. TVM 0.25.0 (official win_amd64 wheel) compiles our ResNet-50 into a module whose output diverges from all other implementations by relative L_2 of 1.24–4.03 with a *different top-1 class*, despite every per-operator micro-test passing at $\sim 10^{-7}$. We isolated a minimal reproduction: any graph containing more than one `BatchNormalization` node is lowered incorrectly, while the imported Relax IR remains correct. Under GATE’s protocol such a plan is rejected by the numerical-error constraint before dispatch; under TVM’s own pipeline it executes silently. We therefore report no latency for the miscompiled module. This incident is precisely the failure mode that motivates validating plans *before* optimizing over them (Section 3)—and it occurred in an off-the-shelf release of a widely deployed compiler.

Scope and threats to validity. The Rust reference prioritizes auditability over speed (no BLAS, no intrinsics); its absolute latency is not competitive with production engines on CPU and we claim no such thing—ONNX Runtime remains $9\times$ faster on this machine. The TPU v5e number runs the same JAX program as the CPU XLA baseline with FP32 matmul precision forced (`jax_default_matmul_precision = highest`); it chiefly demonstrates that the shared-input methodology and the m_5 metric transfer unchanged across a $370\times$ hardware gap. Energy (m_4) on CPU is an analytical estimate (15 W sustained package power $\times$ latency; RAPL counters are not exposed on this Windows platform), and TPU energy is not measurable from a Colab guest. One machine, one model, batch size 1: these results are a proof-of-concept companion to—not a replacement for—the analytical study above.

13 Discussion

13.1 What the Empirical Study Changes

The empirical validation sharpens three claims. First, the cost of formal correctness is smaller than the analytical model conservatively assumed: validation and selection measure in microseconds (Section 12.2, Finding 1), so the earlier “below 3% planning overhead” bound is dominated entirely by optional calibration, not by the algorithm. Second, the risk GATE guards against is demonstrably real: a production compiler release silently miscompiled a standard vision model in a way that per-operator testing did not catch (Finding 3). Third, correctness-first selection has observable consequences (Finding 2), which motivates the normalization discussion below.

13.2 Cost Normalization Sensitivity

Max-normalization makes each metric dimension’s *largest* candidate value the unit. When a dimension’s spread is tiny—e.g. m_5 differing by 10^{-6} between exact and BatchNorm-folded plans—the normalized difference is still 1.0, and the dimension can dominate selection despite being practically insignificant. This is conservative (exactness-preserving) but can forgo real performance. A principled alternative is *threshold-relative* normalization for bounded metrics:

normalize m_5 by the constraint tolerance ϵ rather than the candidate maximum, so deviations far below tolerance contribute proportionally little. We leave a systematic study of normalization schemes to future work; the linear cost model itself (Lemma 4.1) is agnostic to this choice.

13.3 Limitations

- **Candidate-set relativity.** Optimality (Theorem 10.3) is relative to $\mathcal{P}_{\text{cand}}$; heuristic generation may omit the globally optimal strategy.
- **Analytical error model.** Interval-arithmetic bounds may overestimate error and reject acceptable plans; the empirical implementations sidestep this via calibration, which trades planning time for tightness.
- **Single-device execution.** No distributed or multi-GPU execution yet.
- **Backend coverage.** CPU, CUDA, and Vulkan today; TPU access in this paper is via XLA/JAX rather than a native backend.
- **Energy metrology.** m_4 requires platform power counters; on the empirical CPU platform (Windows, no RAPL access) it degrades to an analytical estimate, and cloud TPU guests expose no power telemetry at all. The metric is well-defined but its *measurement* is platform-gated.
- **Reference-implementation performance.** The dependency-free Rust reference is $9\times$ slower than ONNX Runtime on CPU; production deployments would pair GATE with optimized kernels (glproc/glcuda), for which the analytical study models the expected regime.

13.4 Future Work

(1) ML-guided plan generation to improve candidate quality; (2) distributed execution across devices and nodes; (3) runtime constraint adaptation under changing system load; (4) additional backends (native TPU, WebGPU, SYCL); (5) drop-in runtime integration for PyTorch and TensorFlow; (6) formal verification of the constraint implementations themselves; and (7) threshold-relative cost normalization (Section 7).

14 Conclusion

GATE inverts the customary relationship between optimization and correctness in tensor execution: plans are proven valid against seven composable constraint types before any cost is evaluated, and the optimizer’s argmin ranges only over the validated set. We formalized the algorithm, proved constraint soundness, dispatch correctness, and cost optimality, and bounded validation overhead at $O(knp)$.

The empirical study strengthens the case beyond the original analytical projections. Measured on commodity hardware with two independent, fully-executable implementations of the algorithm, the entire validate-and-select step costs microseconds—seven orders of magnitude below one model inference—while five independent implementations across CPU and TPU v5e agree to $\sim 10^{-6}$ relative L_2 on identical inputs. Most tellingly, an off-the-shelf release of a major tensor compiler silently miscompiled ResNet-50 during this very study; the constraint that GATE enforces by construction is the one that catches it.

Correctness in tensor execution does not have to be expensive. It has to be checked.

References

- [1] Dustin B. Apruzzese, Jiong Zhang, Xiaodong Gu, Junjie Bai, Zhijian Yan, Brad Miller, Trishul Chilimbi, and Yutaka Matsuno. ONNX Runtime: A production-ready cross-platform inference engine. 2019.
- [2] Ian A. Buck. XLA: TensorFlow, compiled. In *TensorFlow Dev Summit*. Google, 2017. <https://www.tensorflow.org/xla>.
- [3] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Juan Carlos Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An end to end machine learning compiler framework for CPUs, GPUs, and accelerators. In *Proceedings of the 40th ACM SIGARCH International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 2–13. ACM, 2018.
- [4] Sudheer Chunduri, Andrew Chen, Brian Aein, Cliff Young, and Michael Gschwind. TensorRT: High performance deep learning inference. In *NVIDIA Developer Documentation*. NVIDIA, 2020. <https://developer.nvidia.com/tensorrt>.
- [5] Chris Lattner, Jacques Pienaar, Mehdi Amini, Alex Zinenko, Tatiana Shpeisman, River Riddle, Lei Zhang, Albert Cohen, Andy Davis, Vassil Vassilev, et al. MLIR: A modular infrastructure for compiler development. volume 64, pages 99–109. ACM, 2021.
- [6] George C. Necula. Translation validation for an optimizing compiler. *ACM SIGPLAN Notices*, 35(5):83–94, 2000.
- [7] Georgios P. K. Nikolaidis, Ahmed S. Taei, Stella G. L. Qian, Frank G. Liu, Benoit Keller, T. V. S. R. V. N. Sai, Lei Wang, Yu han Wu, Han Chen, Albert Cohen, et al. IREE: A retargetable compiler and runtime for machine learning. In *Proceedings of the 2nd Workshop on Machine Learning and Systems (MLSys)*, 2022.
- [8] Jens Palsberg and Michael I. Schwartzbach. Translation validation. *Proceedings of the 4th International Conference on Theorem Proving in Higher Order Logics (TPHOLs)*, pages 341–355, 1995.
- [9] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, François-Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *ACM SIGPLAN Notices*, 48(6):519–530, 2013.
- [10] Nadav Rotem, Jade Goldstein, James B. Adkins, Andrew Cumming, Marat Dukhan, Xiaodong Gu, Konstantin Kopylov, Dmitry Lukov, Ivan Pashuk, Nikolai Yakovenko, et al. Glow: Graph lowering compiler techniques for neural networks. 2018.
- [11] Eric Schkufza, Rahul Sharma, and Alex Aiken. Stoke: Synthesis and optimization for superoptimization. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 353–364. ACM, 2013.